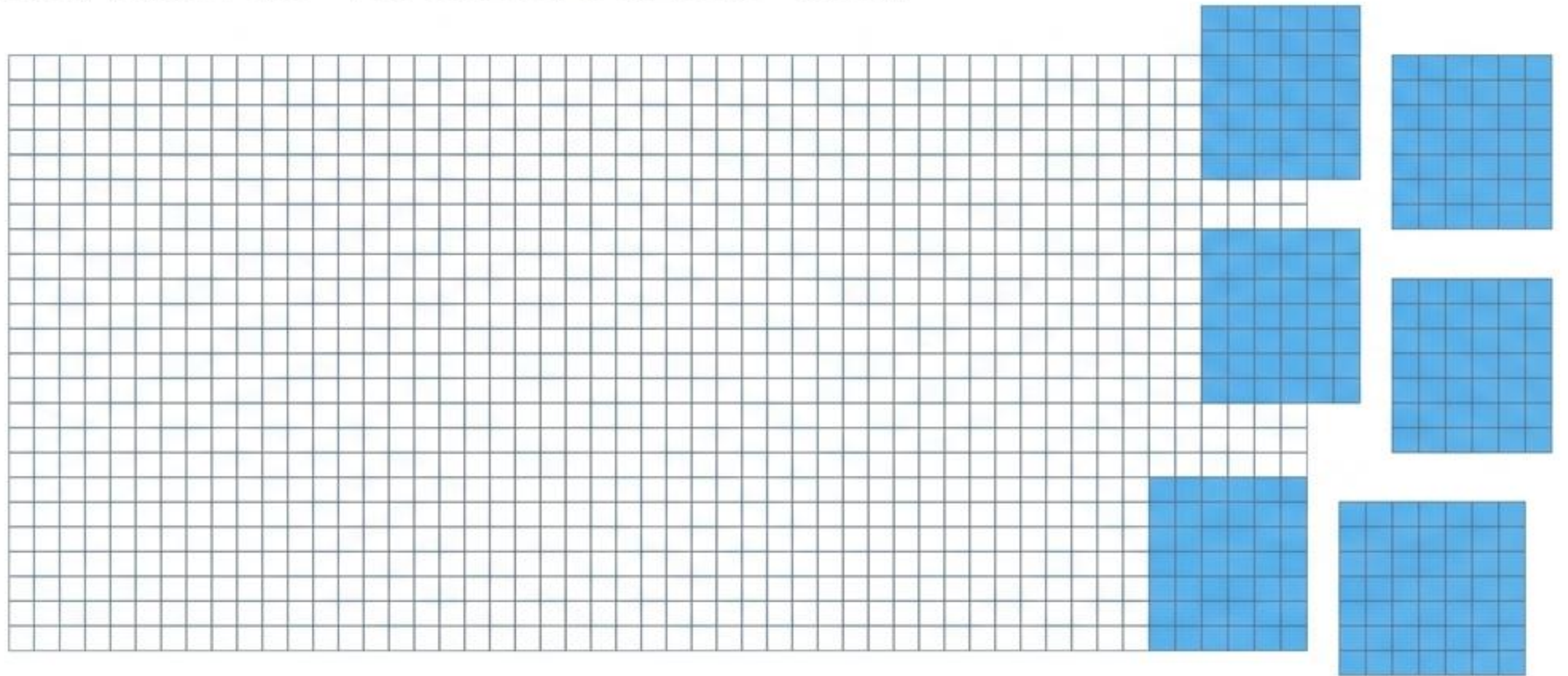


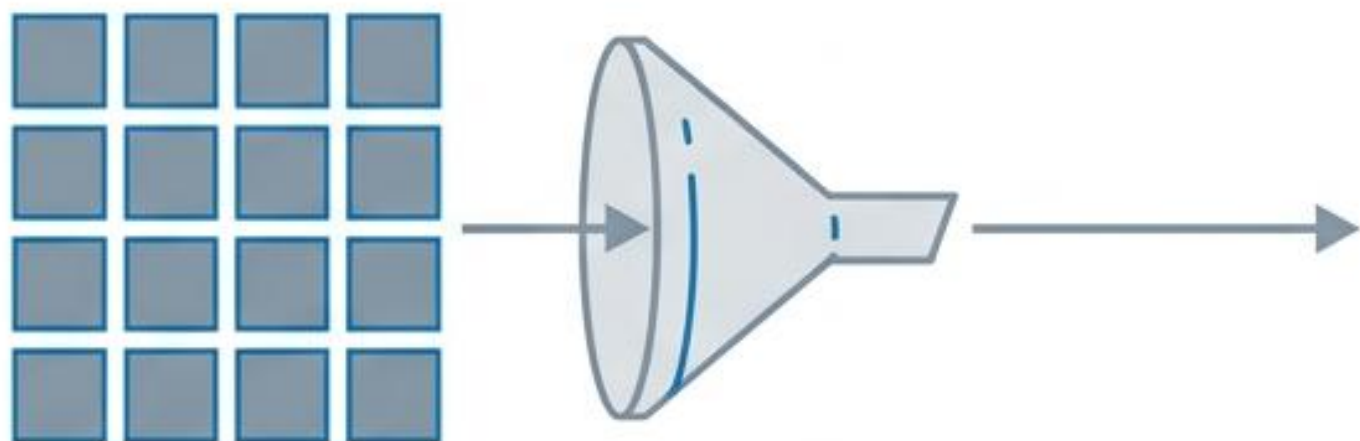
Optimización en Deep Learning

Gradiente Descendente Estocástico con
Mini-batch



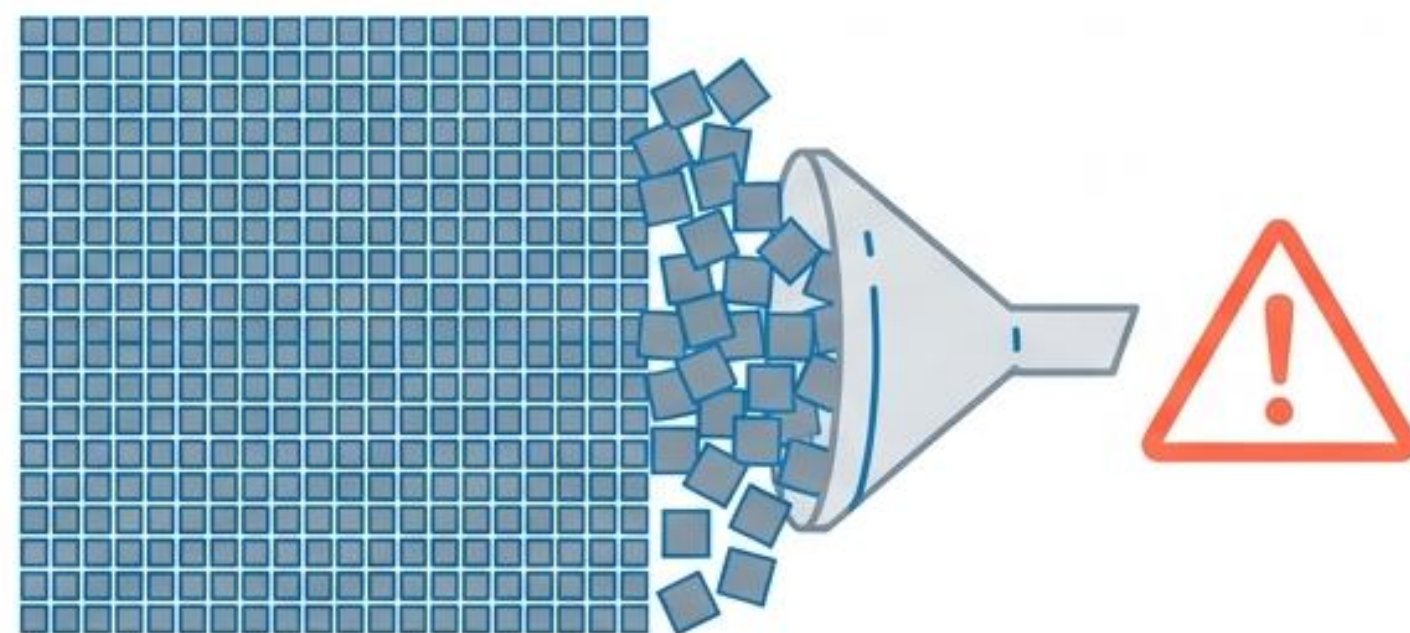
Limitaciones del Gradiente Descendente Batch

Época de pocos datos



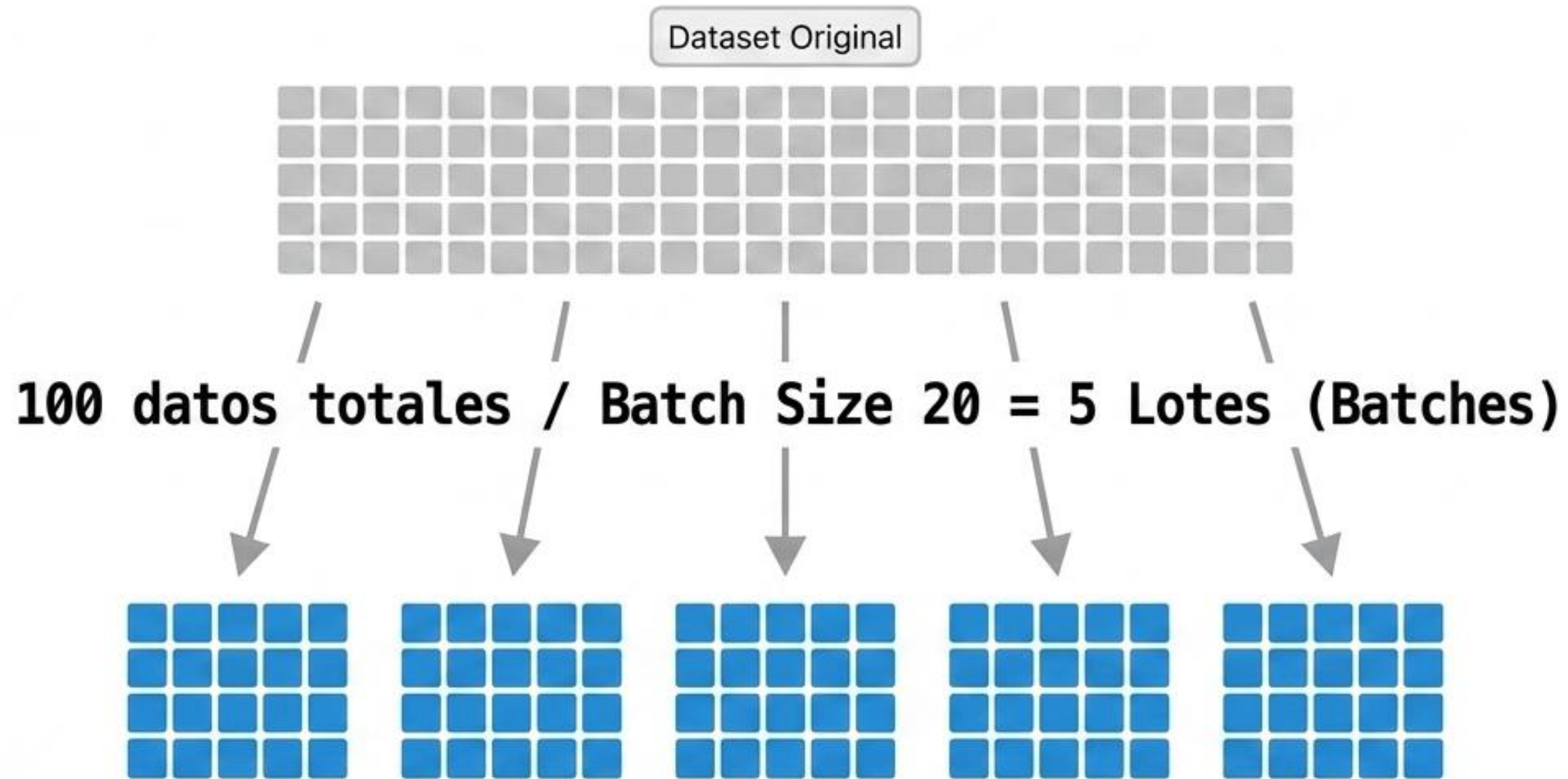
Regresión Lineal Simple: Funciona procesando todos los datos en cada iteración cuando el volumen es bajo.

Deep Learning (Millones de datos)



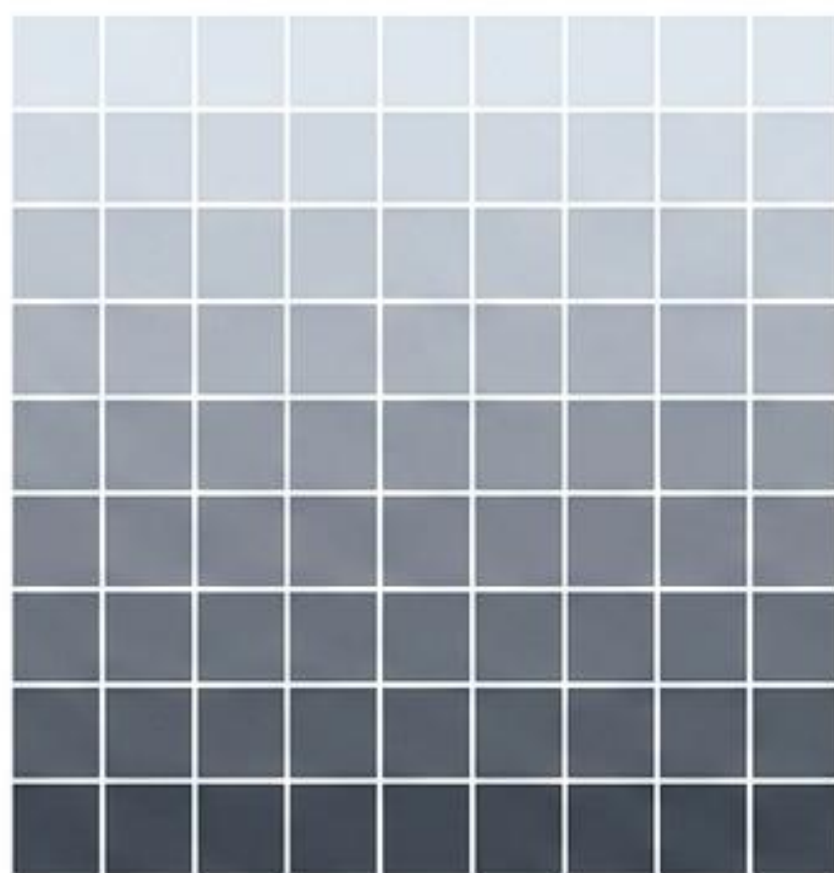
El Cuello de Botella: En Deep Learning, usamos cientos de miles o millones de datos. Evaluar la totalidad del dataset en una sola pasada iterativa colapsa la eficiencia computacional.

Concepto de Mini-batch: Partición del Dataset

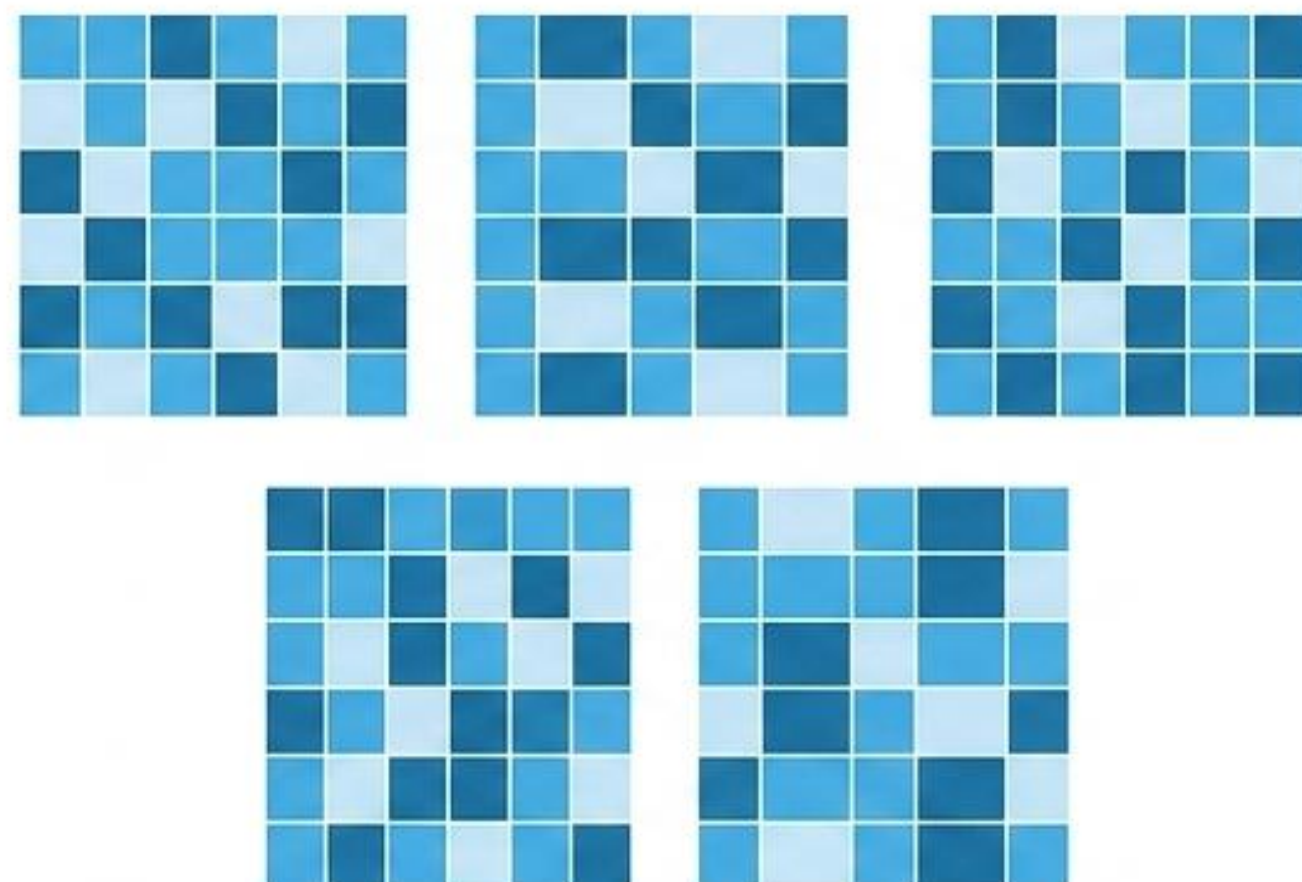
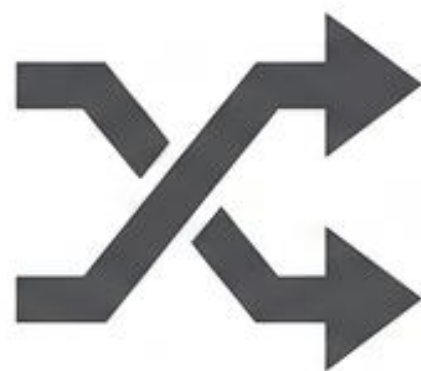


En lugar de presentar todos los datos de golpe en una iteración, el algoritmo divide la carga en grupos o subgrupos más pequeños. Esto permite actualizaciones incrementales y un flujo computacional constante.

La Importancia de la Aleatoriedad



Datos Ordenados



Mini-batches Aleatorios

La aleatoriedad es fundamental para evitar sesgar al modelo. Garantiza que el orden de presentación no afecte el aprendizaje.

Ejemplo de partición aleatoria (Batch Size = 20):
Batch 1: Datos [3, 7, 22, ... 17]
Batch 2: Datos [9, 13, 5, 8, ...]
Batch 5: Datos [6, 100, 12, 97, 80]

Los Tres Hiperparámetros del Algoritmo



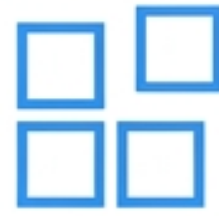
Tasa de
Aprendizaje (α)

El tamaño del paso. Define qué tan drásticamente se ajustan los pesos en cada actualización.



Iteraciones /
Épocas (n_iters)

Número de veces que el algoritmo procesará la totalidad del dataset (todos los lotes).



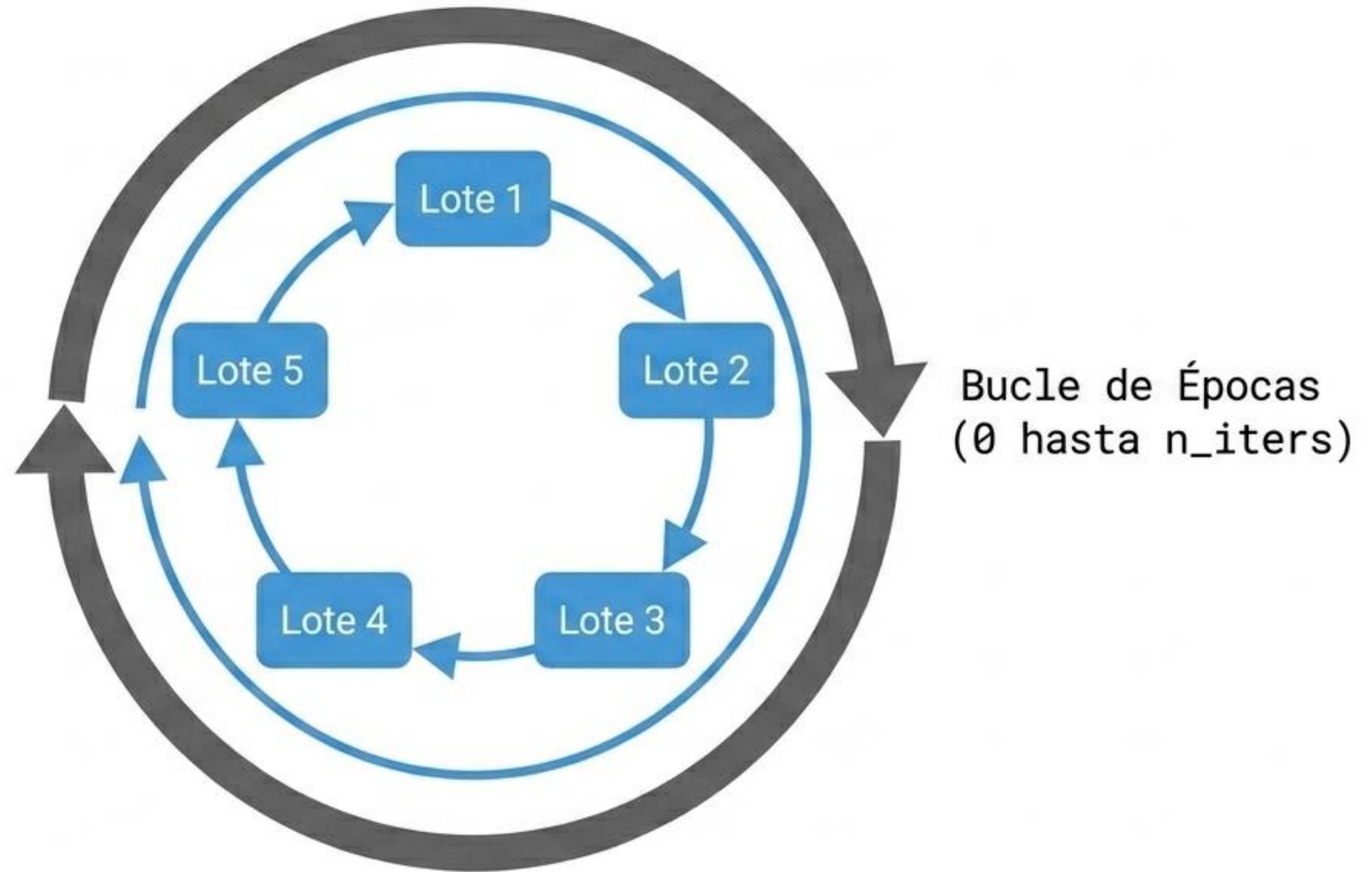
Tamaño del Lote
(batch_size)

El nuevo hiperparámetro. La cantidad exacta de datos seleccionados aleatoriamente que componen un mini-batch.

Lógica del Algoritmo: El Doble Ciclo

El entrenamiento opera sobre dos bucles anidados (for loops):

1. Ciclo Externo: Itera sobre el número total de épocas (n_iters).
2. Ciclo Interno: Toma el dataset dividido en lotes aleatorios y los procesa uno por uno secuencialmente antes de avanzar a la siguiente época.



Actualización de Pesos y Sesgos

$$W = W - \alpha * \nabla L$$

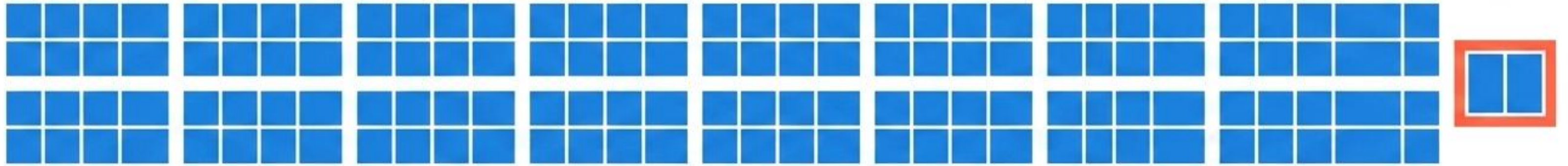
(Donde ∇L es la derivada de la pérdida calculada solo con los datos de este lote específico)

En cada iteración del bucle interno, los parámetros del modelo se actualizan aplicando el gradiente descendente. Se toma el peso anterior, restando la tasa de aprendizaje por la derivada de la pérdida respecto a los pesos (W) o el sesgo (B).

Clave: Si hay 5 lotes, habrá 5 actualizaciones de pesos por cada época.

Datos Restantes: El Manejo de Lotes Incompletos

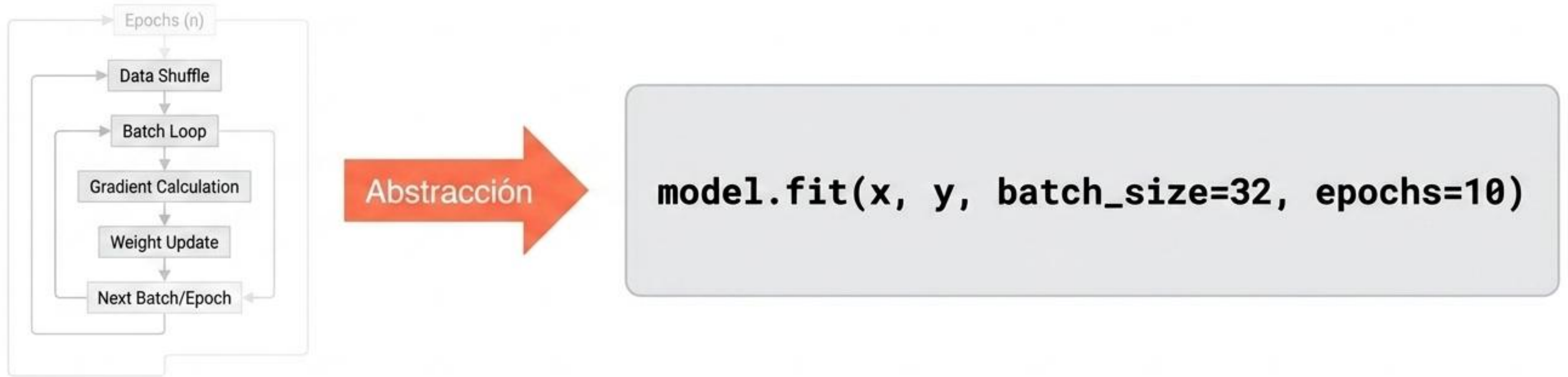
100 datos totales / Batch Size 7 = 14 Lotes Completos + 1 Lote Final de 2 Datos



¿Qué ocurre si el tamaño del dataset no es divisible entre el tamaño del lote? El algoritmo crea los lotes completos posibles y agrupa el resto en un último lote más pequeño.

Regla de Oro: Siempre se garantiza que la totalidad de los datos queden incluidos y procesados en cada época, sin descartar información.

Conclusión: El Estándar de la Industria



Toda la lógica de doble ciclo, aleatorización y manejo de lotes incompletos está abstraída en el ecosistema moderno. Librerías como Keras convierten el Gradiente Descendente Estocástico con Mini-batch en el estándar por defecto para entrenar modelos de Deep Learning.